PROJECT REPORT

FOR

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

CRIME IQ - AI-POWERED CRIMINAL INVESTIGATION GAME

A PROJECT REPORT

SUBMITTED BY

R SNEHA RAJU - 2116240701519

SUBHALAKSHMI M - 2116240701539



OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report entitled **"CRIME IQ - AI-POWERED CRIMINAL INVESTIGATION GAME"** is the bonafide work of **R SNEHA RAJU (2116240701519)** and **SUBHALAKSHMI M (2116240701539)**, who carried out the project work under the guidance and supervision of the faculty in charge.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on: _____

SIGNATURE OF THE INTERNAL EXAMINER

ABSTRACT

CrimeIQ is an interactive, AI-powered criminal investigation game designed to demonstrate the practical application of classical Artificial Intelligence (AI) techniques in problem-solving and decision-making. The game presents fictional crime cases in which players examine suspect profiles, evidence, witness statements and case details to evaluate possible explanations and determine the most likely culprit based on the information available.

The system integrates five classical AI techniques: Iterative Deepening Search (IDS), A* Search, Constraint Satisfaction Problem (CSP) solving with backtracking, propositional logic, and forward chaining. These techniques illustrate systematic exploration of investigative paths, prioritisation of promising possibilities, evaluation of constraints, and derivation of conclusions from facts and rules. Interactive case files, suspect and evidence cards, timelines, algorithm visualisations, and scoring help players connect their decisions with the underlying AI concepts.

CrimeIQ is implemented using HTML5, CSS3 and vanilla JavaScript, with Node.js providing a lightweight local HTTP server. HTML Canvas, SVG and the Web Audio API support visual and interactive effects. The project aims to bridge the gap between theoretical AI concepts and practical understanding through experiential learning. It is an educational simulation rather than a real-world law-enforcement system, and it encourages logical reasoning, exploration of alternatives and evidence-based decision-making.

Keywords: Artificial Intelligence, Crime Investigation Game, Decision-Making, Search Algorithms, Constraint Satisfaction, Propositional Logic, Forward Chaining.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	
1	INTRODUCTION	
1.1	PROBLEM STATEMENT	
1.2	OBJECTIVES	
1.3	SCOPE OF THE PROJECT	
2	LITERATURE REVIEW	
2.1	CLASSICAL AI FOR SEARCH AND REASONING	
2.2	RESEARCH GAP	
3	PROPOSED SYSTEM	
3.1	SYSTEM ARCHITECTURE	
3.2	SYSTEM WORKFLOW	
4	MODULES DESCRIPTION	
4.1	CASE MANAGEMENT MODULE	
4.2	SUSPECT AND EVIDENCE ANALYSIS MODULE	
4.3	ITERATIVE DEEPENING SEARCH MODULE	
4.4	A* SEARCH MODULE	
4.5	CONSTRAINT SATISFACTION AND BACKTRACKING MODULE	
4.6	PROPOSITIONAL LOGIC MODULE	
4.7	FORWARD CHAINING MODULE	
4.8	GAME INTERACTION AND SCORING MODULE	
5	IMPLEMENTATION AND RESULTS	
5.1	DEVELOPMENT ENVIRONMENT AND TECHNOLOGY STACK	
5.2	ALGORITHM IMPLEMENTATION	
5.3	FUNCTIONAL VALIDATION AND OBSERVATIONS	
6	CONCLUSION AND FUTURE ENHANCEMENTS	
7	REFERENCES	

CHAPTER 1
INTRODUCTION

Artificial Intelligence provides computational methods for exploring possibilities, representing knowledge, applying rules and supporting decisions. Crime investigation offers a useful educational setting for these concepts because an investigator must examine incomplete information, compare competing explanations and decide which lead or conclusion is best supported by the available evidence. CrimeIQ uses fictional cases to create a safe, interactive environment for learning these processes.

Unlike a conventional quiz that asks learners to recall definitions, CrimeIQ places the player inside an investigation scenario. The player reviews case information, inspects evidence and suspect details, considers relationships between clues, and makes choices that affect progress and feedback. The algorithms are incorporated as learning demonstrations within the game, showing how different AI approaches can address different reasoning tasks.

1.1 PROBLEM STATEMENT

Classical AI algorithms can be difficult to understand when taught only through equations or isolated code examples. Learners may know the names of search and reasoning methods but struggle to see how they differ or when each method is useful. A unified interactive learning experience is needed to demonstrate these techniques in a meaningful scenario while keeping the focus on structured reasoning and decision-making.

1.2 OBJECTIVES

- To develop an interactive crime-investigation game using fictional cases and evidence.
- To demonstrate IDS, A* Search, CSP solving with backtracking, propositional logic and forward chaining.
- To encourage systematic exploration, constraint-based evaluation and logical deduction.
- To provide an engaging interface with case files, suspect and evidence cards, timelines, algorithm visualisations and scoring.
- To connect theoretical AI concepts with practical decision-making through experiential learning.

1.3 SCOPE OF THE PROJECT

The current project is an educational simulation based on predefined fictional cases and implemented classical algorithms. It is intended to teach AI concepts and does not claim to predict real criminal behaviour, establish guilt in real cases or replace professional investigation. The scope includes case presentation, clue inspection, algorithm demonstrations, player interaction and scoring. Future work may introduce more cases, richer constraints, adaptive difficulty and additional visual explanations.

CHAPTER 2
LITERATURE REVIEW

Research in classical Artificial Intelligence provides the foundations for the search and reasoning methods used in CrimeIQ. Search algorithms model problem solving as exploration of a state space, while logic-based approaches represent facts and derive consequences from them. Constraint satisfaction methods are useful when a solution must obey a set of relationships or restrictions. These ideas are relevant to an educational investigation game because the player must navigate possibilities and judge whether a conclusion is consistent with the available clues.

2.1 CLASSICAL AI FOR SEARCH AND REASONING

Russell and Norvig describe search as a central approach to solving problems in AI, distinguishing uninformed search from informed methods that use estimates to guide exploration. Iterative deepening combines the low memory use of depth-first search with the completeness of breadth-first search for suitable finite branching spaces. A* Search evaluates a candidate state using $f(n) = g(n) + h(n)$, combining the path cost already incurred with a heuristic estimate of the remaining cost. Its optimality depends on assumptions about the heuristic and problem formulation.

Constraint satisfaction problems represent variables, domains and constraints. Backtracking search assigns values incrementally and reverses choices when they lead to inconsistency; forward checking can prune values that are no longer viable. Propositional logic represents facts and their relationships using logical connectives, while forward chaining repeatedly applies inference rules to known facts to derive additional conclusions. These methods are established foundations for structured problem solving and rule-based reasoning.

2.2 RESEARCH GAP

Many educational explanations present search, logic and constraint satisfaction as separate topics. For beginners, it can be difficult to connect the algorithmic process to a concrete decision-making task. CrimeIQ addresses this learning-oriented gap by integrating multiple classical techniques into one interactive fictional investigation environment. The contribution is the combined educational experience and algorithm visualisation, rather than a claim that these techniques alone constitute a production-grade criminal investigation system.

The literature reviewed here supports the theoretical foundations of the implemented methods. It should not be interpreted as evidence that CrimeIQ uses unrelated approaches such as Bayesian networks, automated forensic image analysis or machine-learning-based suspect identification; these are outside the current implementation described in this report.

CHAPTER 3
PROPOSED SYSTEM

CrimeIQ is designed as a browser-based interactive game with a clear separation between the user interface, fictional case data, AI algorithm logic and game interaction. The player selects a case and reviews the information presented through suspect profiles, evidence cards, witness statements and timelines. Depending on the scenario, the game demonstrates search, constraint solving or logical inference, then returns the result to the interface for player interpretation and scoring.

The system is built with HTML5 for semantic structure, CSS3 for styling and visual presentation, and vanilla JavaScript for application logic and algorithm implementation. A lightweight Node.js server uses built-in modules to serve the local application. Canvas-based effects, SVG elements and procedural Web Audio API effects support the themed interactive experience. The implementation does not depend on a frontend framework or a third-party algorithm library.

3.1 SYSTEM ARCHITECTURE

The architecture separates presentation from case logic and algorithm execution. The interface displays the case and collects player actions. Case data provides the suspects, evidence and scenario rules. The algorithm layer contains the implemented search and reasoning techniques. The game interaction layer manages progression, decisions and scoring, while the learning outcome is supported by feedback and algorithm visualisation.

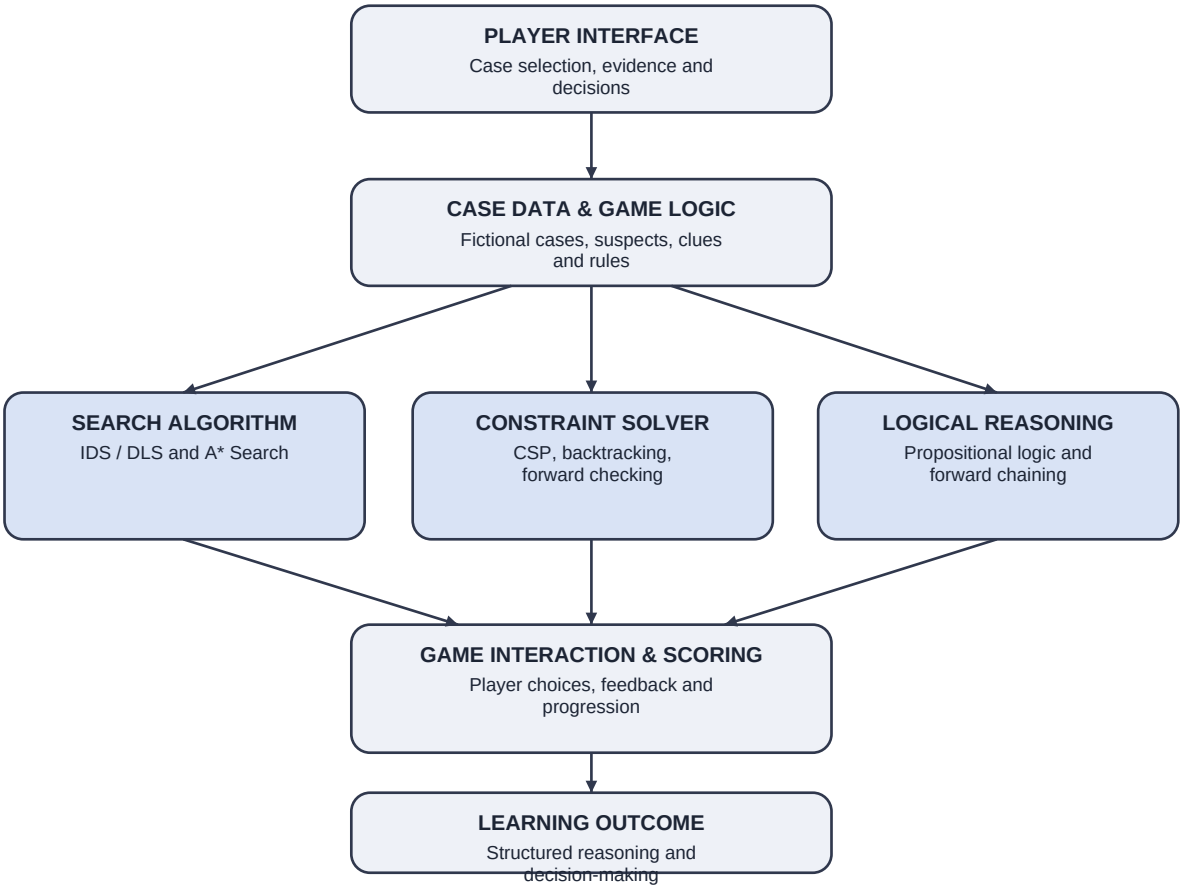


Figure 3.1: HIGH-LEVEL ARCHITECTURE OF CRIMEIQ

3.2 SYSTEM WORKFLOW

The typical workflow begins when a player opens a fictional case. The player reads the case brief, inspects the available clues and compares suspect information. The relevant AI method is used to explore paths, test constraints or derive conclusions from facts and rules. The game presents the resulting information, after which the player makes an investigative decision and receives feedback or a score.

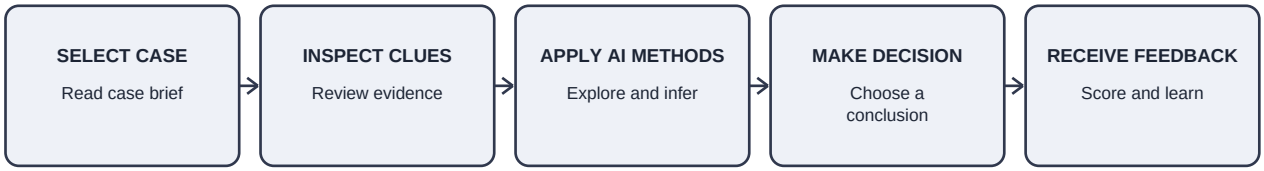


Figure 3.2: PLAYER WORKFLOW IN CRIMEIQ

CHAPTER 4
MODULES DESCRIPTION

CrimeIQ is organised into cooperating modules so that case content, algorithm logic and presentation can be maintained independently. The following descriptions reflect the project's reported implementation and educational purpose.

4.1 CASE MANAGEMENT MODULE

The case management module loads and presents fictional investigation scenarios. Case files contain the narrative context and structured information needed for the player to begin an investigation. Examples in the project include the “Locked Study” and “Vanishing Hour” cases. Keeping case content separate from general interface behaviour makes it easier to extend the game with additional scenarios.

4.2 SUSPECT AND EVIDENCE ANALYSIS MODULE

This module presents the information needed for player reasoning through suspect cards, evidence cards, witness statements and scene details. The information is designed to be reviewed and compared rather than treated as an automatic declaration of guilt. The player uses the presented clues and relationships to form and assess possible explanations.

4.3 ITERATIVE DEEPENING SEARCH MODULE

The IDS implementation combines depth-limited search with repeated increases in the depth limit. Each iteration explores paths up to the current limit; if the goal is not found, the limit is increased and the search restarts. This demonstrates systematic exploration while retaining a memory advantage over breadth-first search in many tree-search settings. The module makes the search process easier to understand by exposing the stages of exploration.

4.4 A* SEARCH MODULE

A* Search ranks states using $f(n) = g(n) + h(n)$, where $g(n)$ represents the known cost from the start and $h(n)$ estimates the remaining cost to the goal. It is useful for demonstrating informed search and the role of a heuristic. The quality and guarantees of A* depend on the problem representation and properties of the heuristic; it should not be assumed to produce a meaningful investigative conclusion without appropriate case-state definitions.

4.5 CONSTRAINT SATISFACTION AND BACKTRACKING MODULE

The CSP module represents a problem using variables, possible domains and constraints. Backtracking assigns a candidate value and checks whether the partial assignment remains consistent. If a conflict appears, the solver returns to an earlier choice and tries another value. Forward checking narrows the remaining domains after assignments, helping demonstrate how constraints can eliminate incompatible combinations.

4.6 PROPOSITIONAL LOGIC MODULE

The propositional logic module represents case statements as propositions and uses logical relationships to evaluate whether conclusions follow from specified premises. Logical connectives allow facts to be combined into rules or conditions. The educational emphasis is on making assumptions explicit and checking whether a proposed conclusion is supported by the encoded statements.

4.7 FORWARD CHAINING MODULE

Forward chaining is a data-driven inference method. It begins with known facts, checks which rules have satisfied premises, and adds their conclusions to the known fact set. The process continues until no new facts can be inferred or a stopping condition is reached. In CrimeIQ, this demonstrates how a sequence of rule applications can produce a conclusion from case facts.

4.8 GAME INTERACTION AND SCORING MODULE

The interaction layer connects the case content with player actions and feedback. It includes investigation screens, visual representations such as timelines and boards, algorithm visualisation and scoring or progression screens. Canvas, SVG and Web Audio API features contribute to the visual and audio presentation. Scoring is a game-learning mechanism and should not be interpreted as a validated measure of real investigative performance.

CHAPTER 5
IMPLEMENTATION AND RESULTS

5.1 DEVELOPMENT ENVIRONMENT AND TECHNOLOGY STACK

The application is implemented with standard web technologies. HTML5 provides the page structure; CSS3 defines layout, typography and visual effects; and vanilla JavaScript implements the interaction logic and classical AI algorithms. The project uses native JavaScript rather than a frontend framework. A Node.js server built with the standard http, fs and path modules serves the application locally. Canvas 2D supports animated effects, SVG supports scalable visual elements and filters, and the Web Audio API supports procedural audio. Google Fonts is loaded through a CDN for typography.

The reported project structure includes separate JavaScript files for IDS/DLS, A* Search, CSP, propositional logic and forward chaining, alongside case files and visual-effect code. The codebase has no external Node.js package dependencies according to the supplied project audit. Git and GitHub are used for version control.

5.2 ALGORITHM IMPLEMENTATION

The algorithms are implemented in native JavaScript and integrated into the game's investigation flow. IDS/DLS supports bounded exploration, A* demonstrates heuristic-guided search, CSP uses backtracking and forward checking to test combinations, propositional logic represents relationships between statements, and forward chaining applies inference rules to known facts. Each algorithm has its own implementation file, supporting modularity and easier demonstration.

A general A* evaluation function is expressed as:

f(n) = g(n) + h(n)

Here, g(n) is the cost accumulated from the start state and h(n) is an estimate of the remaining cost. For CrimeIQ, the actual meaning of a state, action, cost and heuristic depends on the case or demonstration in which the algorithm is used.

5.3 FUNCTIONAL VALIDATION AND OBSERVATIONS

The project audit confirms the presence of separate implementations for the five named classical AI techniques and interactive game components for cases, suspects, evidence, timelines, algorithm visualisation and scoring. These provide the basis for functional demonstrations of search, constraint handling and rule-based inference. The modular design also supports adding or revising fictional cases without requiring a wholesale change to the presentation layer.

This report does not provide fabricated benchmark timings, accuracy percentages or comparative performance figures. A formal quantitative evaluation would require a documented test set, defined expected outputs, repeatable test procedures and recorded results for each algorithm. Future testing should check edge cases such as missing clues, contradictory constraints, unreachable goals, empty fact sets and rules that do not generate new conclusions.

The expected educational value is that learners can observe how different AI methods approach different forms of reasoning. Search methods explore states or paths; CSP checks combinations against restrictions; propositional logic represents explicit relationships; and forward chaining derives additional facts from rules. These observations describe the design intent and should not be treated as measured evidence of improved

learning until a user study is conducted.

CHAPTER 6
CONCLUSION AND FUTURE ENHANCEMENTS

CrimeIQ brings five classical AI techniques together in an interactive, fictional crime-investigation game. Through case files, suspect and evidence cards, witness statements, timelines, algorithm visualisations and scoring, it provides a practical context for exploring search, constraint satisfaction and logical inference. The project is built with HTML5, CSS3, vanilla JavaScript and a lightweight Node.js server, with Canvas, SVG and Web Audio API features enhancing the interface.

The main contribution is educational: the game helps connect abstract algorithmic concepts with a scenario that requires players to inspect information, compare possibilities and make decisions. CrimeIQ is not intended to establish guilt in real cases or to replace professional investigative procedures. Any claim about improved learning outcomes or algorithmic performance would require formal evaluation beyond the implementation description.

Future enhancements may include:

- Adding more fictional cases with varied difficulty and richer evidence relationships.
- Expanding algorithm visualisations to show state expansion, heuristic values, constraint pruning and rule firing step by step.
- Adding hints, guided explanations and post-case feedback tied to the selected AI method.
- Creating structured automated tests for each algorithm and each case scenario.
- Evaluating usability and learning outcomes through a small, documented student study.
- Improving accessibility, responsiveness and localisation of the interface.
- Exploring advanced AI methods as optional future modules while clearly distinguishing them from the current classical-algorithm implementation.

CHAPTER 7
REFERENCES

[1] Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson.

[2] Hart, P. E., Nilsson, N. J. and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100–107.

[3] Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1), pp. 97–109.

[4] Mackworth, A. K. (1977). Consistency in networks of relations. *Artificial Intelligence*, 8(1), pp. 99–118.

[5] Poole, D. L. and Mackworth, A. K. (2017). *Artificial Intelligence: Foundations of Computational Agents*. 2nd ed. Cambridge University Press.

[6] Nilsson, N. J. (1998). *Artificial Intelligence: A New Synthesis*. Morgan Kaufmann.

[7] Aho, A. V., Lam, M. S., Sethi, R. and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson. (Background reference for formal representations and rule processing.)

[8] CrimeIQ project source code and technical audit notes supplied by the project team, October 2026. Used for implementation-specific details and module names.